

102. Binary Tree Level Order Traversal

Problem Summary

Given the root of a binary tree, return the node values level by level as a list of lists.

Approach

BFS using a queue. Process nodes level by level — snapshot the queue size at the start of each level, poll exactly that many nodes, collect their values into a list, then add children for the next level.

Algo

1. If root == null return empty result.
2. Add root to queue.
3. While queue not empty:
 - Snapshot levelSize = q.size().
 - Loop levelSize times: poll curr, add curr.val to level, add children to queue.
 - Add level to result.
4. Return result.

Observations

- No GROUP BY needed — we want a single row output
- Subquery calculates max earnings independently → used in WHERE to filter
- MAX() and COUNT(*) together on filtered rows gives both answers in one query

Key Insights

- levelSize = q.size() snapshot is the key trick — it freezes the count of nodes at the current level before children are added.
- Without the snapshot, you can't tell where one level ends and the next begins.
- Queue naturally maintains left-to-right order within each level.

Time Complexity & Space Complexity

O(n) — every node visited exactly once.

O(n) — queue holds at most one full level; worst case is last level with n/2 nodes.

102. Binary Tree Level Order Traversal

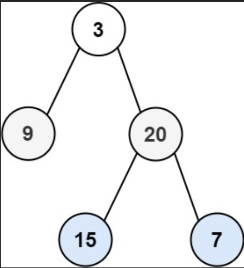
Solved

Medium

TopicsCompaniesHint

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Accepted

36 / 35 testcases passed

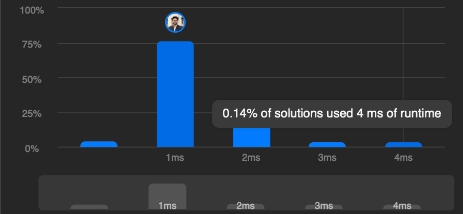
Sahil Khan submitted at Jul 21, 2026 11:11

Runtime

1 ms | Beats 95.87%

Memory

46.78 MB | Beats 62.30%



0.14% of solutions used 4 ms of runtime

Code

Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
10 */
11 class Solution {
12     public List<List<Integer>> levelOrder(TreeNode root) {
13         List<List<Integer>> result = new ArrayList<>();
14
15         if(root == null) return result;
16
17         Queue<TreeNode> q = new LinkedList<>();
18         q.offer(root);
19
20         while(!q.isEmpty()){
21             int levelSize = q.size();
22             List<Integer> level = new ArrayList<>();
23
24             for(int i=0; i<levelSize; i++){
25                 TreeNode curr = q.poll();
26                 level.add(curr.val);
27
28                 if(curr.left != null) q.offer(curr.left);
29                 if(curr.right != null) q.offer(curr.right);
30             }
31
32             result.add(level);
33         }
34
35         return result;
36     }
37 }
```

Testcase

Test Result

You must run your code first